

Introduction to Data Analysis Tools

CPE 311 Computational Thinking with Python
Prepared by Engr. Roman M. Richard



Hands-On Data Analysis with Pandas

Analyze data efficiently for carrying out scientific computing, time series analysis and data visualization using Python

Packt>
www.packt.com

Stefanie Molin

With content from **Hands-on
Data Analysis with Pandas**
by Stefanie Molin

<https://stefaniemolin.com/books/Hands-On-Data-Analysis-with-Pandas-1st-edition/>

Section 1: Getting Started with Pandas

We will begin by introducing the `Series`, `DataFrame`, and `Index` classes, which are the basic building blocks of the pandas library, and showing how to work with them. By the end of this section, you will be able to create DataFrames and perform operations on them to inspect and filter the data.





Outline

1. Anatomy of a DataFrame
2. Creating DataFrames
3. Inspecting the Data
4. Extracting subsets
5. Calculating summary statistics



Anatomy of a Data Frame

A **DataFrame** is composed of one or more **Series**. The names of the **Series** form the column names, and the row labels form the **Index**.

```
In [1]: import pandas as pd

meteorites = pd.read_csv('../data/Meteorite_Landings.csv', nrows=5)
meteorites
```

```
Out[1]:
```

	name	id	nametype	recclass	mass (g)	fall	year	reclat	reclong	GeoLocation
0	Aachen	1	Valid	L5	21	Fell	01/01/1880 12:00:00 AM	50.77500	6.08333	(50.775, 6.08333)
1	Aarhus	2	Valid	H6	720	Fell	01/01/1951 12:00:00 AM	56.18333	10.23333	(56.18333, 10.23333)
2	Abee	6	Valid	EH4	107000	Fell	01/01/1952 12:00:00 AM	54.21667	-113.00000	(54.21667, -113.0)
3	Acapulco	10	Valid	Acapulcoite	1914	Fell	01/01/1976 12:00:00 AM	16.88333	-99.90000	(16.88333, -99.9)
4	Achiras	370	Valid	L6	780	Fell	01/01/1902 12:00:00 AM	-33.16667	-64.95000	(-33.16667, -64.95)

Series:

```
In [2]: meteorites.name
```

```
Out[2]: 0      Aachen  
        1      Aarhus  
        2      Abee  
        3  Acapulco  
        4    Achiras  
        Name: name, dtype: object
```

Columns:

```
In [3]: meteorites.columns
```

```
Out[3]: Index(['name', 'id', 'nametype', 'recclass', 'mass (g)', 'fall', 'year',  
              'reclat', 'reclong', 'GeoLocation'],  
             dtype='object')
```

Index:

```
In [4]: meteorites.index
```

```
Out[4]: RangeIndex(start=0, stop=5, step=1)
```



Creating DataFrames

We can create DataFrames from a variety of sources such as other Python objects, flat files, webscraping, and API requests. Here, we will see just a couple of examples, but be sure to check out [this page](#) in the documentation for a complete list.

[Getting started](#)[User Guide](#)[API reference](#)

pandas documentation

Date: Sep 20, 2024 **Version:** 2.2.3

Download documentation: [Zipped HTML](#)

Previous versions: Documentation of previous versions

Useful links: [Binary Installers](#) | [Source Repository](#)

pandas is an open source, BSD-licensed library providing high performance, [Python](#) data analysis tools for the [Python](#) programming language.

Using a flat file

```
In [5]: import pandas as pd

meteorites = pd.read_csv('../data/Meteorite_Landings.csv')
```

Using data from an API

Collect the data from [NASA's Open Data Portal](#) using the Socrata Open Data API (SODA) with the `requests` library:

```
In [6]: import requests

response = requests.get(
    'https://data.nasa.gov/resource/gh4g-9sfh.json',
    params={'$limit': 50_000}
)

if response.ok:
    payload = response.json()
else:
    print(f'Request was not successful and returned code: {response.status_code}.')
    payload = None
```

We won't be using this today. But for information...

Create the DataFrame with the resulting payload:

In [7]:

```
import pandas as pd

df = pd.DataFrame(payload)
df.head(3)
```

Out [7]:

	name	id	nametype	recclass	mass	fall	year	reclat	reclong	geolocation	:@computed_region_cbhk_fw
0	Aachen	1	Valid	L5	21	Fell	1880-01-01T00:00:00.000	50.775000	6.083330	{'latitude': '50.775', 'longitude': '6.08333'}	Na
1	Aarhus	2	Valid	H6	720	Fell	1951-01-01T00:00:00.000	56.183330	10.233330	{'latitude': '56.18333', 'longitude': '10.23333'}	Na
2	Abee	6	Valid	EH4	107000	Fell	1952-01-01T00:00:00.000	54.216670	-113.000000	{'latitude': '54.21667', 'longitude': '-113.0'}	Na



Inspecting Data

Now that we have some data, we need to perform an initial inspection of it. This gives us information on what the data looks like, how many rows/columns there are, and how much data we have.

Let's inspect the **meteorites data**.

How many rows and columns are there?

```
In [8]: meteorites.shape
```

```
Out[8]: (45716, 10)
```

What are the column names?

```
In [9]: meteorites.columns
```

```
Out[9]: Index(['name', 'id', 'nametype', 'recclass', 'mass (g)', 'fall', 'year',  
              'reclat', 'reclong', 'GeoLocation'],  
             dtype='object')
```

What type of data does each column currently hold?

```
In [10]: meteorites.dtypes
```

```
Out[10]: name          object  
         id            int64
```

What does the data look like?

```
In [11]: meteorites.head()
```

```
Out[11]:
```

	name	id	nametype	recclass	mass (g)	fall	year	reclat	reclong	GeoLocation
0	Aachen	1	Valid	L5	21.0	Fell	01/01/1880 12:00:00 AM	50.77500	6.08333	(50.775, 6.08333)
1	Aarhus	2	Valid	H6	720.0	Fell	01/01/1951 12:00:00 AM	56.18333	10.23333	(56.18333, 10.23333)
2	Abee	6	Valid	EH4	107000.0	Fell	01/01/1952 12:00:00 AM	54.21667	-113.00000	(54.21667, -113.0)
3	Acapulco	10	Valid	Acapulcoite	1914.0	Fell	01/01/1976 12:00:00 AM	16.88333	-99.90000	(16.88333, -99.9)
4	Achiras	370	Valid	L6	780.0	Fell	01/01/1902 12:00:00 AM	-33.16667	-64.95000	(-33.16667, -64.95)

Sometimes there may be extraneous data at the end of the file, so checking the bottom few rows is also important:

```
In [12]: meteorites.tail()
```

```
Out[12]:
```

	name	id	nametype	recclass	mass (g)	fall	year	reclat	reclong	GeoLocation
45711	Zillah 002	31356	Valid	Eucrite	172.0	Found	01/01/1990 12:00:00 AM	29.03700	17.01850	(29.037, 17.0185)
45712	Zinder	30409	Valid	Pallasite, ungrouped	46.0	Found	01/01/1999 12:00:00 AM	13.78333	8.96667	(13.78333, 8.96667)
45713	Zlin	30410	Valid	H4	3.3	Found	01/01/1939 12:00:00 AM	49.25000	17.66667	(49.25, 17.66667)
45714	Zubkovsky	31357	Valid	L6	2167.0	Found	01/01/2003 12:00:00 AM	49.78917	41.50460	(49.78917, 41.5046)
45715	Zulu Queen	30414	Valid	L3.7	200.0	Found	01/01/1976 12:00:00 AM	33.98333	-115.68333	(33.98333, -115.68333)

Get some information about the DataFrame

In [13]:

```
meteorites.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 45716 entries, 0 to 45715
```

```
Data columns (total 10 columns):
```

#	Column	Non-Null Count	Dtype
0	name	45716 non-null	object
1	id	45716 non-null	int64
2	nametype	45716 non-null	object
3	recclass	45716 non-null	object
4	mass (g)	45585 non-null	float64
5	fall	45716 non-null	object
6	year	45425 non-null	object
7	reclat	38401 non-null	float64
8	reclong	38401 non-null	float64
9	GeoLocation	38401 non-null	object

```
dtypes: float64(3), int64(1), object(6)
```

```
memory usage: 3.5+ MB
```



Extracting Subsets

A crucial part of working with DataFrames is extracting subsets of the data: finding rows that meet a certain set of criteria, isolating columns/rows of interest, etc. After narrowing down our data, we are closer to discovering insights. This section will be the backbone of many analysis tasks.

Selecting columns

We can select columns as attributes if their names would be valid Python variables:

```
In [16]: meteorites.name
```

```
Out[16]: 0          Aachen
1          Aarhus
2           Abee
3        Acapulco
4        Achiras
...
45711    Zillāh 002
45712         Zinder
45713         Zlin
45714    Zubkovsky
45715    Zulu Queen
Name: name, Length: 45716, dtype: object
```

If they aren't, we have to select them as keys. However, we can select multiple columns at once this way:

```
In [17]: meteorites[['name', 'mass (g)']]
```

```
Out[17]:
```

	name	mass (g)
0	Aachen	21.0
1	Aarhus	720.0
2	Abee	107000.0
3	Acapulco	1914.0
4	Achiras	780.0
...	...	...
45711	Zillah 002	172.0
45712	Zinder	46.0
45713	Zlin	3.3
45714	Zubkovsky	2167.0
45715	Zulu Queen	200.0

45716 rows × 2 columns

Selecting rows

```
In [18]: meteorites[100:104]
```

```
Out[18]:
```

	name	id	nametype	recclass	mass (g)	fall	year	reclat	reclong	GeoLocation
100	Benton	5026	Valid	LL6	2840.0	Fell	01/01/1949 12:00:00 AM	45.95000	-67.55000	(45.95, -67.55)
101	Berduc	48975	Valid	L6	270.0	Fell	01/01/2008 12:00:00 AM	-31.91000	-58.32833	(-31.91, -58.32833)
102	Béréba	5028	Valid	Eucrite-mmict	18000.0	Fell	01/01/1924 12:00:00 AM	11.65000	-3.65000	(11.65, -3.65)
103	Berlanguillas	5029	Valid	L6	1440.0	Fell	01/01/1811 12:00:00 AM	41.68333	-3.80000	(41.68333, -3.8)

Indexing

We use `iloc[]` to select rows and columns by their position:

```
In [19]: meteorites.iloc[100:104, [0, 3, 4, 6]]
```

```
Out[19]:
```

	name	recclass	mass (g)	year
100	Benton	LL6	2840.0	01/01/1949 12:00:00 AM
101	Berduc	L6	270.0	01/01/2008 12:00:00 AM
102	Béréba	Eucrite-mmict	18000.0	01/01/1924 12:00:00 AM
103	Berlanguillas	L6	1440.0	01/01/1811 12:00:00 AM

We use `loc[]` to select by name:

```
In [20]: meteorites.loc[100:104, 'mass (g)':'year']
```

```
Out[20]:
```

	mass (g)	fall	year
100	2840.0	Fell	01/01/1949 12:00:00 AM
101	270.0	Fell	01/01/2008 12:00:00 AM
102	18000.0	Fell	01/01/1924 12:00:00 AM
103	1440.0	Fell	01/01/1811 12:00:00 AM
104	960.0	Fell	01/01/2004 12:00:00 AM



Filtering with Boolean masks

A **Boolean mask** is a array-like structure of Boolean values – it's a way to specify which rows/columns we want to select (**True**) and which we don't (**False**).

Here's an example of a Boolean mask for meteorites weighing more than 50 grams that were found on Earth (i.e., they were not observed falling):

```
In [21]: (meteorites['mass (g)'] > 50) & (meteorites.fall == 'Found')
```

```
Out[21]: 0      False
          1      False
          2      False
          3      False
          4      False
          ...
          45711   True
          45712   False
          45713   False
          45714   True
          45715   True
          Length: 45716, dtype: bool
```

Important: Take note of the syntax here. We surround each condition with parentheses, and we use bitwise operators (&, |, ~) instead of logical operators (and, or, not).

We can use a Boolean mask to select the subset of meteorites weighing more than 1 million grams (1,000 kilograms or roughly 2,205 pounds) that were observed falling:

```
In [22]: meteorites[(meteorites['mass (g)'] > 1e6) & (meteorites.fall == 'Fell')]
```

```
Out[22]:
```

	name	id	nametype	recclass	mass (g)	fall	year	reclat	reclong	GeoLocation
29	Allende	2278	Valid	CV3	2000000.0	Fell	01/01/1969 12:00:00 AM	26.96667	-105.31667	(26.96667, -105.31667)
419	Jilin	12171	Valid	H5	4000000.0	Fell	01/01/1976 12:00:00 AM	44.05000	126.16667	(44.05, 126.16667)
506	Kunya-Urgench	12379	Valid	H5	1100000.0	Fell	01/01/1998 12:00:00 AM	42.25000	59.20000	(42.25, 59.2)
707	Norton County	17922	Valid	Aubrite	1100000.0	Fell	01/01/1948 12:00:00 AM	39.68333	-99.86667	(39.68333, -99.86667)
920	Sikhote-Alin	23593	Valid	Iron, IIAB	23000000.0	Fell	01/01/1947 12:00:00 AM	46.16000	134.65333	(46.16, 134.65333)

Tip: Boolean masks can be used with `loc[]` and `iloc[]`.

An alternative to this is the `query()` method:

```
In [23]: meteorites.query("`mass (g)` > 1e6 and fall == 'Fell'")
```

Out[23]:	name	id	nametype	recclass	mass (g)	fall	year	reclat	reclong	GeoLocation
29	Allende	2278	Valid	CV3	2000000.0	Fell	01/01/1969 12:00:00 AM	26.96667	-105.31667	(26.96667, -105.31667)
419	Jilin	12171	Valid	H5	4000000.0	Fell	01/01/1976 12:00:00 AM	44.05000	126.16667	(44.05, 126.16667)
506	Kunya-Urgench	12379	Valid	H5	1100000.0	Fell	01/01/1998 12:00:00 AM	42.25000	59.20000	(42.25, 59.2)
707	Norton County	17922	Valid	Aubrite	1100000.0	Fell	01/01/1948 12:00:00 AM	39.68333	-99.86667	(39.68333, -99.86667)
920	Sikhote-Alin	23593	Valid	Iron, IIAB	23000000.0	Fell	01/01/1947 12:00:00 AM	46.16000	134.65333	(46.16, 134.65333)

Tip: Here, we can use both logical operators and bitwise operators.



Calculating Summary Statistics

In this section, we will discuss data cleaning for a more meaningful analysis of our datasets; however, we can already extract some interesting insights from the `meteorites` data by calculating summary statistics.

How many of the meteorites were found versus observed falling?

```
In [24]: meteorites.fall.value_counts()
```

```
Out[24]: fall
Found      44609
Fell        1107
Name: count, dtype: int64
```

The Meteoritical Society states that "relict meteorites are composed mostly of terrestrial minerals, but are thought to have once been meteorites." What proportion of the data are relict meteorites? Let's verify these were all found versus observed falling:

```
In [25]: meteorites.value_counts(subset=['nametype', 'fall'], normalize=True)
```

```
Out[25]:
```

nametype	fall
Valid	Found 0.974145
	Fell 0.024215
Relict	Found 0.001641

Name: proportion, dtype: float64

What was the mass of the average meteorite?

```
In [26]: meteorites['mass (g)'].mean()
```

```
Out[26]: np.float64(13278.078548601512)
```

Important: The mean isn't always the best measure of central tendency. If there are outliers in the distribution, the mean will be skewed. Here, the mean is being pulled higher by some very heavy meteorites – the distribution is [right-skewed](#).

Taking a look at some quantiles at the extremes of the distribution shows that the mean is between the 95th and 99th percentile of the distribution, so it isn't a good measure of central tendency here:

```
In [27]: meteorites['mass (g)'].quantile([0.01, 0.05, 0.5, 0.95, 0.99])
```

```
Out[27]: 0.01      0.44
0.05      1.10
0.50     32.60
0.95    4000.00
0.99   50600.00
Name: mass (g), dtype: float64
```

A better measure in this case is the median (50th percentile), since it is robust to outliers:

```
In [28]: meteorites['mass (g)'].median()
```

```
Out[28]: np.float64(32.6)
```

What was the mass of the heaviest meteorite?

```
In [29]: meteorites['mass (g)'].max()
```

```
Out[29]: np.float64(60000000.0)
```

Let's extract the information on this meteorite:

```
In [30]: meteorites.loc[meteorites['mass (g)'].idxmax()]
```

```
Out[30]:
```

name	Hoba
id	11890
nametype	Valid
recclass	Iron, IVB
mass (g)	60000000.0
fall	Found
year	01/01/1930 12:00:00 AM

How many different types of meteorite classes are represented in this dataset?

```
In [31]: meteorites.recclass.nunique()
```

```
Out[31]: 466
```

Some examples:

```
In [32]: meteorites.recclass.unique()[14]
```

```
Out[32]: array(['L5', 'H6', 'EH4', 'Acapulcoite', 'L6', 'LL3-6', 'H5', 'L',  
                'Diogenite-pm', 'Unknown', 'H4', 'H', 'Iron, IVA', 'CR2-an'],  
               dtype=object)
```

Note: All fields preceded with "rec" are the values recommended by The Meteoritical Society. Check out [this Wikipedia article](#) for some information on meteorite classes.

Get some summary statistics on the data itself

We can get common summary statistics for all columns at once. By default, this will only be numeric columns, but here, we will summarize everything together:

In [33]: `meteorites.describe(include='all')`

Out[33]:

	name	id	nametype	recclass	mass (g)	fall	year	reclat	reclong	GeoLocation
count	45716	45716.000000	45716	45716	4.558500e+04	45716	45425	38401.000000	38401.000000	38401
unique	45716	NaN	2	466	NaN	2	266	NaN	NaN	17100
top	Aachen	NaN	Valid	L6	NaN	Found	01/01/2003 12:00:00 AM	NaN	NaN	(0.0, 0.0)
freq	1	NaN	45641	8285	NaN	44609	3323	NaN	NaN	6214
mean	NaN	26889.735104	NaN	NaN	1.327808e+04	NaN	NaN	-39.122580	61.074319	NaN
std	NaN	16860.683030	NaN	NaN	5.749889e+05	NaN	NaN	46.378511	80.647298	NaN
min	NaN	1.000000	NaN	NaN	0.000000e+00	NaN	NaN	-87.366670	-165.433330	NaN
25%	NaN	12688.750000	NaN	NaN	7.200000e+00	NaN	NaN	-76.714240	0.000000	NaN
50%	NaN	24261.500000	NaN	NaN	3.260000e+01	NaN	NaN	-71.500000	35.666670	NaN
75%	NaN	40656.750000	NaN	NaN	2.026000e+02	NaN	NaN	0.000000	157.166670	NaN
max	NaN	57458.000000	NaN	NaN	6.000000e+07	NaN	NaN	81.166670	354.473330	NaN

Important: NaN values signify missing data. For instance, the `fall` column contains strings, so there is no value for `mean`; likewise, `mass (g)` is numeric, so we don't have entries for the categorical summary statistics (`unique`, `top`, `freq`).



Exercise (Part 1)

1. Create a DataFrame by reading in the `2019_Yellow_Taxi_Trip_Data.csv` file. Examine the first 5 rows.
2. Find the dimensions (number of rows and number of columns) in the data.
3. Using the data in the `2019_Yellow_Taxi_Trip_Data.csv` file, calculate summary statistics for the `fare_amount`, `tip_amount`, `tolls_amount`, and `total_amount` columns.
4. Isolate the `fare_amount`, `tip_amount`, `tolls_amount`, and `total_amount` for the longest trip by distance (`trip_distance`).

Exercise (Part 1)

1. Create a DataFrame by reading in the `2019_Yellow_Taxi_Trip_Data.csv` file. Examine the first 5 rows.

In [14]:

```
import pandas as pd

taxi = pd.read_csv('../data/2019_Yellow_Taxi_Trip_Data.csv')
taxi.head()
```

Out[14]:

	vendorid	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	ratecodeid	store_and_fwd_flag	pulocationid
0	2	2019-10-23T16:39:42.000	2019-10-23T17:14:10.000	1	7.93	1	N	
1	1	2019-10-23T16:32:08.000	2019-10-23T16:45:26.000	1	2.00	1	N	
2	2	2019-10-23T16:08:44.000	2019-10-23T16:21:11.000	1	1.36	1	N	
3	2	2019-10-23T16:22:44.000	2019-10-23T16:43:26.000	1	1.00	1	N	
4	2	2019-10-23T16:45:11.000	2019-10-23T16:58:49.000	1	1.96	1	N	

Exercise (Part 1)

2. Find the dimensions (number of rows and number of columns) in the data.

```
In [15]: taxi.shape
```

```
Out[15]: (10000, 18)
```

Exercise (Part 1)

3. Using the data in the `2019_Yellow_Taxi_Trip_Data.csv` file, calculate summary statistics for the `fare_amount`, `tip_amount`, `tolls_amount`, and `total_amount` columns.

```
In [34]: import pandas as pd

taxi = pd.read_csv('../data/2019_Yellow_Taxi_Trip_Data.csv')
taxi[['fare_amount', 'tip_amount', 'tolls_amount', 'total_amount']].describe()
```

```
Out[34]:
```

	fare_amount	tip_amount	tolls_amount	total_amount
count	10000.000000	10000.000000	10000.000000	10000.000000
mean	15.106313	2.634494	0.623447	22.564659
std	13.954762	3.409800	6.437507	19.209255
min	-52.000000	0.000000	-6.120000	-65.920000
25%	7.000000	0.000000	0.000000	12.375000
50%	10.000000	2.000000	0.000000	16.300000
75%	16.000000	3.250000	0.000000	22.880000
max	176.000000	43.000000	612.000000	671.800000

Exercise (Part 1)



4. Isolate the fare_amount, tip_amount, tolls_amount, and total_amount for the longest trip by distance (trip_distance).

```
In [35]: taxi.loc[
            taxi.trip_distance.idxmax(),
            ['fare_amount', 'tip_amount', 'tolls_amount', 'total_amount']]
```

```
Out[35]: fare_amount      176.0
          tip_amount      18.29
          tolls_amount      6.12
          total_amount    201.21
          Name: 8338, dtype: object
```

